MODULE $EWD687a$

This is an algorithm for detecting termination in distributed computations by *Edsger Dijkstra* and *Carel Scholten*. It originally appeared in *Dijkstra*'s note $EWD687a$, available at

https://*www.cs.utexas.edu*/users/*EWD*/transcriptions/*EWD*06*xx*/*EWD*687*a.html*

and which was published as

"Termination Detection for Diffusing Computations" in Inf. Process. Lett., 11(1) (Aug. 1980) : $1 - 4$.

The terminology in this specification comes not from that version, but from the description of the algorithm in Section 4.5.2 of the forthcoming book *ACM* book about *Dijkstra*, where it is called the "Tree Algorithm". (See *EWD*840.*tla* at https://*git.io*/*JDBfG* for the TLA+ specification of the "Ring Algorithm" described in that book, and *DijkstraScholten.tla* at https://*git.io*/*JDBvx* for a *PlusCal* variant of the tree algorithm.)

We assume a network of processes that perform a computation. What they computed is irrelevant for this specification. All we care is that each process is either idle or busy computing. A busy process can send messages to other processes. Receipt of a message makes a process busy; a busy process may become idle at any time. Initially, only a single process called the leader is busy; all other processes are idle. The leader starts the distributed computation by sending messages to some of the processes of the network. We want an algorithm that informs the leader when the computation has terminated, which is when all processes are idle and there are no messages in transit.

The network is described by a directed graph having the processes as its nodes. A process p can send messages to a process q if and only if there is an edge from p to q. We assume there are no edges pointing to the leader, so it can send messages but not receive them. The termination-detection algorithm may introduce additional control messages, which are distinct from the computation's messages. If there is an edge from process p to process q, then we assume control messages can be sent from q to p in addition to the computation messages sent from p to q.

Suppose that the graph describing the network is a directed tree having the leader as its root. That means that it has no cycles and there is a unique path from the leader to each node. If there is an edge from process p to process q, we say that p is the parent of q and q is a child of p. The descendants of a process are the nodes reachable from that process. For such a graph, we can use the following algorithm. For each message a child receives from its parent, it must eventually send a control message, called an acknowledgment (ack), to its parent. A process can send an ack to its parent any time after it receives the corresponding message, with one restriction: It can send the last ack (for the messages it has received thus far) to its parent only when it is idle and it has received *acks* for all the messages it has sent to its children. Define a process to be neutral if it is idle and has sent *acks* to its parent for every message it has received (from its parent). An induction argument shows that, for such a graph, the algorithm maintains the following invariant: if a node is neutral, then all its descendants are neutral. From this it follows that, when the leader is neutral, all processes are neutral, which implies that all processes are idle and all computation messages have been received, so the computation has terminated.

The basic idea of the Dijkstra-Scholten Tree Algorithm is to handle an arbitrary network by applying the algorithm described above to a dynamically constructed "overlay" tree. In an arbitrary network, a process can receive messages from more than one other process. It sends an ack to the sender of each message it receives. We define a process to be neutral if it is idle, has sent *acks* for every message it has received, and has received *acks* for every message it has sent. Initially, every process but the leader is neutral. The overlay tree is constructed by making the process from which a neutral process p receives a message (making p non-neutral) the parent of p. Process p can send its last ack to its parent only when doing so makes it neutral–that is, only when it is idle, has received *acks* for all messages it has sent, and has sent *acks* for all messages it has received except for one of the messages it has received from its parent. The algorithm maintains the invariant that all non-neutral processes form a tree rooted at the leader. Thus, when the leader is neutral, all other processes are also neutral and the computation has terminated.

The algorithm allows asynchronous, unordered message delivery, but it assumes (computation and control) messages are not lost. Non-Byzantine process failures cannot cause the leader to believe that a non-terminated computation has terminated, but it can cause termination not to be detected. It requires that a process can send acknowledgment messages to any process that can send it computation messages.

EXTENDS *Integers*

We declare the following constants:

Procs: The set of processes.

Leader: The leader process.

Edges: The set of edges in the process graph, where an edge from process p to process q is the pair $\langle p, q\rangle$. The element $\langle p, q\rangle$ being in *Edges* means that p can send computation messages to q. Note that we cannot model redundant edges between p and q, since $\{\langle p, q\rangle, \langle p, q\rangle\} = \{\langle p, q\rangle\}$.

CONSTANTS *Procs*, *Leader*, *Edges*

InEdges(p) is the set of edges pointing to process p. That is, it is the set of all pairs $\langle q, p\rangle$ such that q can send a computation message to p.

$$InEdges(p) \triangleq \{e \in Edges : e[2] = p\}$$

OutEdges(p) is the set of edges pointing from p. That is, it is the set of all pairs $\langle p, q\rangle$ such that p can send computation messages to q (and q can send *acks* to p).

$$OutEdges(p) \triangleq \{e \in Edges : e[1] = p\}$$

The following assumption asserts what we have stated in the comments about the values of the declared constants *Procs*, *Leader*, and Edges–namely, that an edge is a pair of processes and the leader is a process that can't receive computation messages. It also asserts something that was not stated explicitly: that a process can't send messages to itself, so an edge joins two different processes.

ASSUME $EdgeFacts \triangleq$

$\land$ Every edge is a pair of distinct processes

$\quad \forall\, e \in Edges :$

$\quad\quad \land\, (e \in Procs \times Procs)$

$\quad\quad \land\, (e[1] \neq e[2])$

$\land$ The leader is one of the processes.

$\quad Leader \in Procs$

$\land$ The leader has only outgoing edges, so it can't receive computation messages.

$$InEdges(Leader) = \{\}$$

Our algorithm needs a piece of data that indicates whether a process has a parent in the overlay tree and, if so, who that parent is. We represent that piece of data as the edge from the parent to the process if the process has a parent, and a value *NotAnEdge* if the process has no parent. We can define *NotAnEdge* to be any value that is not an element of the set *Edges* of edges. Since the elements of *Edges* are ordered pairs, we can define *NotAnEdge* to be any value that is not an ordered pair. The simplest such value is the empty sequence (zero-tuple) $\langle\rangle$.

$NotAnEdge \triangleq \langle\rangle$

We declare the following variables:

VARIABLES

For each process p, the value of $active[p]$ is a boolean. If the processes p is currently busy computing, the value of $active[p]$ is true. If process p is idle, the value of $active[p]$ is false. In this spec, we do not care what is computed.

$active$,

For two processes q and p and $\langle q, p\rangle \in Edges$, the value of $sentUnacked[\langle q, p\rangle]$ is the (natural) number of messages sent from q to p that p has yet to acknowledge (by sending a control message to q). In other words, $sentUnacked[\langle q, p\rangle]$ is incremented when q sends a computational message to process p, and decremented when q receives a control message (ack) back from process p.

$sentUnacked$,

The value of $rcvdUnacked[\langle q, p\rangle]$ is the number of messages sent to process p thus far for which q has not yet received a control message from p acknowledging its receipt. Thus, it is the number of *acks* that p still has to send to q.

The value of $rcvdUnacked[\langle q, p\rangle]$ is incremented when p receives a computational message from process q. It is decremented when p sends a control message (ack) to process q.

$rcvdUnacked$,

Let the overlay tree be defined as:

$O \triangleq$
LET $E \triangleq \{upEdge[p] : p \in \text{DOMAIN } upEdge\} \setminus \{NotAnEdge\}$IN
[edges $\mapsto E$, nodes $\mapsto \{e[1] : e \in E\} \cup \{e[2] : e \in E\}$]

(We do not bother to change the direction of the edges in O to point towards the leader.)

For all e in $O.edges$ with $e = \langle q, p\rangle$, the process q is the process that caused p to become non-neutral, which we call the parent of process p.

Process q remains p's parent for as long as p remains non-neutral. When p becomes neutral, it leaves O by setting $upEdge[p]$ to *NotAnEdge*. Thus, $O.nodes$ are always non-neutral process, which is asserted by the property *TreeWithRoot* below.

Generally, O may change many times before the leader detects (global) termination. This can be studied by checking the non-property of this spec *StableUpEdge* below.

If $O.edges$ is unequal to $\{\}$, *i.e.* non-empty, the overlay tree is a a tree rooted in the leader, which is asserted in the property *TreeWithRoot* below.

$upEdge$,

The difference between the variables above and *msgs* & *acks* below is that active, *sentUnacked*, and *rcvdUnacked* are conceptually process-local state, while *msgs* and *acks* are the state of the network. In an implementation, we would, thus, expect to find code that maintains the values of active, *sentUnacked*, and *rcvdUnacked* in the implementation of a process. The variables *msgs* and *acks*, however, would likely be implicitly because they model the state of the network.

For an edge $\langle q, p\rangle \in \mathit{InEdges}(p)$, $\mathit{msgs}[\langle q, p\rangle]$ denotes the number of pending (computational) messages at process p whose sender is process q. In other words, $\mathit{msgs}[\langle q, p\rangle]$ is the number of messages in transit on the edge $\langle q, p\rangle$. Note that the content of messages is irrelevant for this spec. It suffices to keep track of the number of unreceive messages.

msgs,

For an edge $\langle q, p\rangle \in \mathit{OutEdges}(p)$, $\mathit{acks}[\langle q, p\rangle]$ is the number of control) messages in transit from process p to process q.

acks

The fact that we use counters to model *acks* and *msgs* implies that message ordering does not matter for the correctness of this algo.

$$\mathit{vars} \triangleq \langle \mathit{active}, \mathit{msgs}, \mathit{acks}, \mathit{sentUnacked}, \mathit{rcvdUnacked}, \mathit{upEdge}\rangle$$

TypeOK asserts the "types" of all variables. The value of variable active is in the set of functions with domain *Procs* and co-domain BOOLEAN . The values of *msgs*, *acks*, *sentUnacked*, *rcvdUnacked* are in the set of functions with with domain *Edges* and co-domain Naturals. The value of *upEdge* is in the set of functions with domain the set of (non-leader) processes and co-domain the union of *Edges* and $\{\mathit{NotAnEdge}\}$. If $\mathit{upEdge}[p]$ for a process p is unequal to *NotAnEdge*, that $\mathit{upEdge}[p]$ is an edge in *Edges*. Moreover, that edge is joining some (other) process to process p.

$$\begin{aligned}\mathit{TypeOK} \triangleq\ & \land\ \mathit{active} \in [\mathit{Procs} \to \text{BOOLEAN }]\\ & \land\ \mathit{msgs} \in [\mathit{Edges} \to \mathit{Nat}]\\ & \land\ \mathit{acks} \in [\mathit{Edges} \to \mathit{Nat}]\\ & \land\ \mathit{sentUnacked} \in [\mathit{Edges} \to \mathit{Nat}]\\ & \land\ \mathit{rcvdUnacked} \in [\mathit{Edges} \to \mathit{Nat}]\\ & \land\ \mathit{upEdge} \in [\mathit{Procs} \setminus \{\mathit{Leader}\} \to \mathit{Edges} \cup \{\mathit{NotAnEdge}\}]\\ & \land\ \forall\, p \in \mathit{Procs} \setminus \{\mathit{Leader}\} :\\ & \qquad \mathit{upEdge}[p] \neq \mathit{NotAnEdge} \Rightarrow \mathit{upEdge}[p][2] = p\end{aligned}$$

A process p is neutral, iff it is idle (not computing) and has received an ack for every message it has sent, and it has sent an ack for every message it has received.

$$\begin{aligned}\mathit{neutral}(p) \triangleq\ & \land\ \neg \mathit{active}[p]\\ & \land\ \forall\, e \in \mathit{InEdges}(p) : \mathit{rcvdUnacked}[e] = 0\\ & \land\ \forall\, e \in \mathit{OutEdges}(p) : \mathit{sentUnacked}[e] = 0\end{aligned}$$

The initial predicate *Init* defines the values of all variables when the execution of this algorithm begins. Initially, all processes except the leader are neutral. No process, not even the leader, has sent a computational message. Therefore, there are also zero pending control messages (ack). For every non-leader p, the initial value of $\mathit{upEdge}[p]$ is equal to *NotAnEdge*. Thus, the overlay tree is empty.

$$\begin{aligned}\mathit{Init} \triangleq\ & \land\ \mathit{active} = [p \in \mathit{Procs} \mapsto p = \mathit{Leader}]\\ & \land\ \mathit{msgs} = [e \in \mathit{Edges} \mapsto 0]\end{aligned}$$

$\land\ acks\ = [e \in Edges \mapsto 0]$
$\land\ sentUnacked = [e \in Edges \mapsto 0]$
$\land\ rcvdUnacked = [e \in Edges \mapsto 0]$
$\land\ upEdge = [p\ \in Procs \setminus \{Leader\} \mapsto NotAnEdge]$

We now define the subactions of the next-state actions. We begin by defining an action that will be used in those subactions. The action *SendMsg*(p) describes the action in which an active process p sends a (computational) message via one of its outgoing edges. Sending a message causes p to expect the receipt of an ack on the same edge on which p sent the (computational) message.

$SendMsg(p) \triangleq \land\ active[p]$
$\land \exists\, e \in OutEdges(p) :$
$\land\ sentUnacked' = [sentUnacked\ \text{EXCEPT}\ ![e] = @ + 1]$
$\land\ msgs' = [msgs\ \text{EXCEPT}\ ![e] = @ + 1]$
$\land\ \text{UNCHANGED}\ \langle active,\ acks,\ rcvdUnacked,\ upEdge\rangle$

The *RcvAck* subaction describes what a process p does when an ack is pending. Process p receives the ack on an edge e *s.t* $e[1] = p$. The receipt of the acknowledgment also makes p decrement the number of expected *acks* on that edge.

$RcvAck(p) \triangleq \exists\, e \in OutEdges(p) :$
$\land\ acks[e] > 0$
$\land\ acks' = [acks\ \text{EXCEPT}\ ![e] = @ - 1]$
$\land\ sentUnacked' = [sentUnacked\ \text{EXCEPT}\ ![e] = @ - 1]$
$\land\ \text{UNCHANGED}\ \langle active,\ msgs,\ rcvdUnacked,\ upEdge\rangle$

The *SendAck* subaction defines how a process p acknowledges the receipt of a message ($rcvdUnacked[e] > 0$) by sending an ack along edge $\langle q, p\rangle$. Process p may send an ack if one or more of the following conditions are true:

1. The receiver q of the ack is not the parent of p in the overlay tree, *i.e.*, $upEdge[p] \neq \langle q, p\rangle$
2*a*. The receiver q of the ack is the parent of p in the overlay tree, *i.e.*, $upEdge[p] = \langle q, p\rangle$, but the parent q is expecting more than one ack from p.
2*b*. The receiver q of the ack is the parent of p in the overlay tree, and p is neutral after sending the ack.

UP. If process p is neutral after sending an ack (hence $netural(p)'$), it removes itself from the overlay tree by setting the value of $upEdge[p]$ to *NotAnEdge*.

$SendAck(p) \triangleq \land \exists\, e \in InEdges(p) :$
$\land\ rcvdUnacked[e] > 0$
1. q *not* the parent of p (intentionally vacuously true).
$\land\ (e = upEdge[p]) \Rightarrow$
2*a*. q parent and q is expecting more than one ack from p.
$\lor\ rcvdUnacked[e] > 1$
2*b*. q parent and p is neutral after sending the ack
($neutral(p)'$ implied by $neutral(p)$ while e is ignored).
$\lor\ \land \neg active[p]$

$$
\begin{array}{l}
\quad \land \forall\, d \in \mathit{InEdges}(p) \setminus \{e\} : \mathit{rcvdUnacked}[d] = 0 \\
\quad \land \forall\, d \in \mathit{OutEdges}(p) : \mathit{sentUnacked}[d] = 0 \\
\quad \text{Observe the similarity of the above three conjuncts with } \mathit{neutral}(p) \text{ above.} \\
\land\ \mathit{rcvdUnacked}' = [\mathit{rcvdUnacked} \text{ EXCEPT } ![e] = @ - 1] \\
\land\ \mathit{acks}' = [\mathit{acks} \text{ EXCEPT } ![e] = @ + 1] \\
\land \text{ UNCHANGED } \langle \mathit{active},\ \mathit{msgs},\ \mathit{sentUnacked} \rangle \\
\text{Note that no check for } p = \mathit{Leader} \text{ is needed because the} \\
\text{leader never sends } \mathit{acks}, \text{ due to } \mathit{InEdges}(\mathit{Leader}) = \{\} \text{ .} \\
\land\ \mathit{UP}!: \mathit{upEdge}' = \text{IF } \mathit{neutral}(p)' \text{ THEN } [\mathit{upEdge} \text{ EXCEPT } ![p] = \mathit{NotAnEdge}] \\
\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \text{ELSE } \mathit{upEdge}
\end{array}
$$

The subaction *RcvMsg* describes what a process p does when it receives a message. Assuming a pending computational message on edge e
($\mathit{msgs}[e] > 0$), process p re-activates upon receipt of this message
(if p was already active, the message is still received without changing p's active state).

Moreover, if process p is neutral at the time when it receives the message, the edge e becomes p's *upEdge*. In other words, process $\mathit{upEdge}[p][1]$ becomes the parent of process p in the overlay tree.

$$
\begin{array}{l}
\mathit{RcvMsg}(p) \triangleq \exists\, e \in \mathit{InEdges}(p) : \\
\quad \land\ \mathit{msgs}[e] > 0 \\
\quad \land\ \mathit{msgs}' = [\mathit{msgs} \text{ EXCEPT } ![e] = @ - 1] \\
\quad \land\ \mathit{rcvdUnacked}' = [\mathit{rcvdUnacked} \text{ EXCEPT } ![e] = @ + 1] \\
\quad \land\ \mathit{active}' = [\mathit{active} \text{ EXCEPT } ![p] = \text{TRUE}] \\
\quad \text{If the process } p \text{ is neutral, process q becomes the} \\
\quad \text{parent of } p \text{ in the overlay tree.} \\
\quad \land\ \mathit{upEdge}' = \text{IF } \mathit{neutral}(p) \text{ THEN } [\mathit{upEdge} \text{ EXCEPT } ![p] = e] \\
\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \text{ELSE } \mathit{upEdge} \\
\quad \land \text{ UNCHANGED } \langle \mathit{acks},\ \mathit{sentUnacked} \rangle
\end{array}
$$

A process p may finish its computation and become idle at any time.

If a non-leader process p is neutral after an idle step, it implies that p was not a node of the overlay tree when it became idle. Thus, there is no need to change $\mathit{upEdge}[p]$ in the Idle subaction.

$$
\begin{array}{l}
\mathit{Idle}(p) \triangleq \land\ \mathit{active}' = [\mathit{active} \text{ EXCEPT } ![p] = \text{FALSE}] \\
\qquad\qquad \land \text{ UNCHANGED } \langle \mathit{msgs},\ \mathit{acks},\ \mathit{sentUnacked},\ \mathit{rcvdUnacked},\ \mathit{upEdge} \rangle
\end{array}
$$

$$
\begin{array}{l}
\mathit{Next} \triangleq \exists\, p \in \mathit{Procs} : \mathit{SendMsg}(p) \lor \mathit{SendAck}(p) \lor \mathit{RcvMsg}(p) \lor \mathit{RcvAck}(p) \\
\qquad\qquad\qquad\qquad\qquad\qquad\quad \lor \quad \mathit{Idle}(p)
\end{array}
$$

$$
\mathit{Spec} \triangleq \mathit{Init} \land \Box[\mathit{Next}]_{\mathit{vars}} \land \mathrm{WF}_{\mathit{vars}}(\mathit{Next})
$$

EWd687a assumes messages are not lost. Thus, the four counters always have to be consistent, *i.e.*, the sum of $\mathit{msgs}[e]$, $\mathit{acks}[e]$, and $\mathit{rcvdUnacked}[e]$ equals $\mathit{sentUnacked}[e]$, for any e in *Edges*.

$$
\begin{array}{l}
\mathit{CountersConsistent} \triangleq \\
\quad \Box \forall\, e \in \mathit{Edges} : \mathit{sentUnacked}[e] = \mathit{rcvdUnacked}[e] + \mathit{acks}[e] + \mathit{msgs}[e]
\end{array}
$$

THEOREM $Spec \Rightarrow CountersConsistent$

The overlay tree is a tree of non-neutral nodes rooted in the leader, or the tree is empty *s.t.* it has no edges nor nodes. With bigger graphs, expect a significant slowdown when model-checking the property *TreeWithRoot*.

In a variant of this spec, the conjunct labelled Up in the *SendAck* subactions could be removed. In this variant, the processes in the overlay tree would be processes that are or were non-neutral in the past. Consequently, the conjunct labelled N would have to be removed from the property *TreeWithRoot* .

$$
\begin{array}{l}
TreeWithRoot \triangleq \\
\quad \text{LET } E \triangleq \{upEdge[p] : p \in \text{DOMAIN } upEdge\} \setminus \{NotAnEdge\} \\
\qquad N \triangleq \{e[1] : e \in E\} \cup \{e[2] : e \in E\} \\
\qquad G \triangleq \text{INSTANCE } Graphs \\
\qquad O \triangleq G!Transpose([edge \mapsto E,\ node \mapsto N]) \\
\quad \text{IN } \Box(\\
\qquad\quad O \text{ is a tree rooted in the leader.} \\
\qquad\quad \land\ O.edge \neq \{\} \Rightarrow G!IsTreeWithRoot(O,\ Leader) \\
\qquad\quad \text{All nodes of the overlay tree are non-neutral.} \\
\qquad\quad \land\ N :: \forall\, n \in O.node : \neg neutral(n))
\end{array}
$$

THEOREM $Spec \Rightarrow TreeWithRoot$

Main safety property: The leader detects that the computation has terminated only if (global) termination has actually occurred.

$$DT1Inv \triangleq neutral(Leader) \Rightarrow \forall\, p \in Procs \setminus \{Leader\} : neutral(p)$$

THEOREM $Spec \Rightarrow \Box DT1Inv$

The computation has terminated if all processes (including the leader) are idle, and there are no messages pending (globally).

$$
\begin{array}{ll}
Terminated \triangleq & \land \forall\, p \in Procs : \neg active[p] \\
& \land \forall\, e \in Edges : msgs[e] = 0
\end{array}
$$

Main liveness property: If the computation terminates, then the leader eventually detects it.

$$DT2 \triangleq Terminated \leadsto neutral(Leader)$$

THEOREM $Spec \Rightarrow DT2$

Counter-examples to the non-property below can be helpful to understand the algorithm.

$$
\begin{array}{l}
StableUpEdge \triangleq \\
\quad \text{The parent of a process in the overlay tree never changes. This property is not a property of the algorithm, } i.e.\text{, a theorem.} \\
\quad \Box[\forall\, p \in Procs \setminus \{Leader\} :
\end{array}
$$

$$(upEdge[p] \neq NotAnEdge) \Rightarrow upEdge[p] = upEdge'[p]]_{upEdge}$$

* Modification History
* Last modified *Tue Dec* 21 17:52:54 *PST* 2021 by *Markus Kuppe*
* Last modified *Fri Dec* 17 17:35:25 *PST* 2021 by *Leslie Lamport*
* Created *Wed Sep* 29 01:36:40 *PDT* 2021 by *Leslie Lamport*